LLMOps Field Reference

Complete Production Pipeline Guide for Deployed Engineers

Vector Databases • LLM Orchestration • Agentic AI • RAG • Evaluation • Security • LLMOps • Observability

Covering: LangChain • LangGraph • LangSmith • DeepEval • OpenAI SDK • LiteLLM • FAISS • Pinecone • ChromaDB • Weaviate • Qdrant

1. Vector Databases & Semantic Search

1.1 Why Vector Databases?

Traditional keyword search fails at semantic retrieval — it cannot match concepts, synonyms, or intent. Vector databases store dense numerical representations (embeddings) of content and enable similarity-based retrieval using distance metrics.

Core Problem Statement

- TFIDF/BM25 misses: 'heart attack' vs 'myocardial infarction'
- LLMs have limited context windows — you cannot feed all your data every request
- Production RAG systems need millisecond retrieval over millions of documents
- Semantic search enables intent-matching, not just token-matching

1.2 Similarity Search Concepts

Metric	Formula (conceptual)	Use Case	When to Avoid
Cosine Similarity	dot(A,B)/(|A||B|)	Text embeddings (most common)	When magnitude matters
Euclidean (L2)	sqrt(sum((a-b)^2))	Image embeddings, dense vectors	High-dimensional sparse data
Dot Product	sum(a*b)	When vectors are normalized	Unnormalized vectors — misleading
Manhattan (L1)	sum(|a-b|)	Robust to outliers	High-dim general text

1.3 Approximate Nearest Neighbor (ANN)

Exact nearest neighbor search is O(n·d) per query — infeasible at scale. ANN trades a small accuracy drop for massive speed gains. All production vector DBs use ANN internally.

HNSW (Hierarchical Navigable Small World)

- Multi-layer graph structure; upper layers for coarse navigation, lower for fine search
- Parameters: ef_construction (build quality, higher=slower build, better recall), M (connections per node, higher=more memory, better recall), ef (search quality, runtime tunable)
- Typical recall@10: 95-99% at <10ms for 1M vectors

- Best for: real-time search, latency-sensitive use cases
- Avoid: memory-constrained environments (RAM = roughly 100 bytes × M × vectors)

IVF (Inverted File Index)

- Clusters vectors into nlist centroids; at query time searches only nprobe closest clusters
- Parameters: nlist (number of clusters, sqrt(n) is good default), nprobe (how many to search, higher=better recall but slower)
- Requires training phase — not suitable for dynamic/streaming datasets
- Best for: batch workloads, memory-constrained systems, very large static corpora

Algorithm	Build Speed	Query Speed	Memory	Recall	Dynamic Updates
HNSW	Slow	Very Fast	High	Very High	Yes (incremental)
IVF	Medium	Fast	Medium	High	No (rebuild needed)
IVF+PQ	Fast	Very Fast	Low (compressed)	Medium	No
ScaNN (Google)	Medium	Very Fast	Medium	Very High	Limited
Annoy (Spotify)	Fast	Medium	Low	Medium	No

1.4 Vector Database Comparison

DB	Type	ANN	Filtering	Scale	Managed	Best For
Pinecone	Cloud-only	HNSW	Metadata	Billions	Yes	Production, zero-ops
Weaviate	Hybrid	HNSW	Rich GraphQL	100M+	Yes/Self	Multi-modal, complex filters
Qdrant	OSS/Cloud	HNSW	Advanced	100M+	Yes/Self	High perf, payload filters
ChromaDB	OSS	HNSW	Basic	<1M	Self	Dev/local prototyping
FAISS	Library	Multiple	None built-in	Billions	Self	Research, custom pipelines
pgvector	Postgres ext	HNSW/IVF	SQL	<10M	Self	Existing Postgres infra

DB	Type	ANN	Filtering	Scale	Managed	Best For
Redis VectorDB	Cloud/Self	HNSW	Redis filters	10M+	Yes/Self	Low-latency, small corpus

ChromaDB Warning ChromaDB is NOT production-ready for large scale. Use it only in development. For staging/prod use Qdrant (self-hosted) or Pinecone (managed).

1.5 Vector Storage & Retrieval Workflow

Ingestion Pipeline

1. Load raw documents (PDF, HTML, DOCX, web)
2. Chunk documents (fixed, recursive, semantic, or semantic chunking)
3. Generate embeddings (OpenAI text-embedding-3-small/large, BGE, E5, Cohere)
4. Store vectors + metadata in vector DB with document IDs
5. Store raw chunks in document store (S3, Postgres) keyed by same IDs

Retrieval Pipeline

6. Embed incoming query using SAME embedding model as ingestion
7. ANN search in vector DB → top-k chunk IDs + similarity scores
8. Apply metadata filters (date, category, department, access level)
9. Fetch raw text from document store by chunk IDs
10. Pass ranked chunks as context to LLM

ç Critical: Embedding Model Mismatch NEVER change your embedding model after ingestion without re-indexing ALL documents. Different models produce incompatible vector spaces. This is the #1 silent RAG failure mode.

1.6 Chunking Strategy — The Most Underrated Decision

Strategy	Method	Chunk Size	Pros	Cons	Use When
Fixed Size	Split every N chars	256-512 tokens	Simple, fast	Cuts mid-sentence	Quick prototyping only
Recursive	Split on \n\n, \n, space	300-600 tokens	Respects paragraphs	Still arbitrary	General purpose default
Semantic	Embed +	Variable	Topic-	Slow,	Legal,

Strategy	Method	Chunk Size	Pros	Cons	Use When
	cosine boundary		coherent chunks	expensive	scientific docs
Sentence Window	Sentence units + context	1-3 sentences	Precise retrieval	Many small chunks	QA over dense text
Parent-Child	Store parent, retrieve child	Small retrieval, large context	High precision + recall	Complex pipeline	Production RAG

Production Recommendation Use Parent-Child chunking for production: embed small child chunks for precise retrieval, but pass the full parent document chunk to the LLM as context. This maximizes both retrieval precision and generation quality.

1.7 Metadata Filtering

Metadata filtering reduces the search space BEFORE vector similarity, dramatically improving both speed and precision.

- Always store: document_id, source_url, created_at, updated_at, department, doc_type, access_level
- Pre-filter (most DBs): filter applied before ANN search — fast but can reduce recall
- Post-filter: ANN first, then filter — better recall but may need larger k to compensate
- Qdrant supports payload filters natively with no performance penalty
- Pinecone supports metadata filtering with limits on cardinality — avoid high-cardinality keys

1.8 Hybrid Search (Keyword + Vector)

Pure semantic search misses exact matches (product codes, names, IDs). Hybrid search combines BM25 (keyword) and vector (semantic) scores.

Reciprocal Rank Fusion (RRF)

RRF(d) = sum(1 / (k + rank_i(d))) — combines rankings without needing score normalization. k=60 is standard default.

- Weaviate: built-in hybrid search with alpha parameter (0=keyword, 1=vector)
- Qdrant: sparse + dense vector hybrid out of the box
- Elasticsearch: kNN + BM25 hybrid (mature, enterprise-grade)
- Pinecone: not native — use separate BM25 index and merge client-side

When to Use Hybrid Search Always enable hybrid search in production. Pure vector

search fails on: exact product codes, legal clause references, version numbers, people names, acronyms. Hybrid adds <5ms latency for significant precision gains.

1.9 Re-Ranking

Initial retrieval gives top-k candidates. Re-ranking applies a more expensive cross-encoder model to re-score only those k results for better final ordering.

Re-ranker	Type	Speed	Quality	Cost
Cohere Rerank	API	Fast	Excellent	Pay-per-use
BGE-Reranker	Local	Medium	Very Good	Free (GPU needed)
ColBERT	Local	Slow	Excellent	Free (GPU needed)
Cross-Encoder (SBERT)	Local	Slow	Good	Free
LLM-as-Judge	LLM API	Very Slow	Best	Expensive

Re-ranking Pattern Retrieve top 20 → re-rank → pass top 5 to LLM. Never re-rank hundreds of results; the quality gain does not justify the latency. Use Cohere Rerank for managed, BGE-Reranker for self-hosted budget.

1.10 Performance Considerations & Tuning

Bottleneck	Symptom	Solution	Avoid
Embedding generation	Slow ingestion	Batch embed, use async, GPU acceleration	Embedding one doc at a time
ANN search latency	Slow queries	Tune ef, increase replicas, use HNSW	Increasing nprobe without profiling
Memory pressure	OOM errors	Use IVF+PQ compression, shard index	Loading full HNSW into small RAM
Metadata filter + vector	Slow filtered search	Pre-compute filter sets, use Qdrant payload index	Filtering on string columns not indexed
Cold start	Slow first query	Warm-up queries on deploy, keep containers hot	Letting containers spin down (serverless trap)

2. LLM Orchestration — LangChain, LangGraph & Pipelines

2.1 Why Orchestration?

Raw LLM API calls handle single-turn requests. Real applications need: multi-step reasoning, memory across turns, tool usage, retrieval injection, conditional logic, error retry, and streaming. Orchestration frameworks provide these as composable primitives.

2.2 LangChain Architecture Deep Dive

Core Components

Component	Purpose	Key Classes	Production Notes
Prompts	Template + format input	PromptTemplate, ChatPromptTemplate, FewShotPromptTemplate	Always version prompts; avoid f-strings in production
Chains	Compose LLM calls	LLMChain, SequentialChain, RunnableSequence (LCEL)	Use LCEL (pipe syntax) — old Chain API is deprecated
Memory	Persist conversation	ConversationBufferMemory, ConversationSummaryMemory, VectorStoreRetrieverMemory	Never use BufferMemory in prod — unbounded token growth
Tools	External capabilities	Tool, BaseTool, @tool decorator	Always add timeout + error handling to tools
Retrievers	Fetch relevant docs	VectorStoreRetriever, MultiQueryRetriever, EnsembleRetriever	Use EnsembleRetriever for hybrid BM25+vector
Output Parsers	Structure LLM output	PydanticOutputParser, JsonOutputParser, StrOutputParser	Always validate parsed output; never trust raw LLM JSON

LCEL — LangChain Expression Language (Modern API)

LCEL uses the pipe operator (|) to compose Runnables. This is the correct way to build chains in 2024+. Old LLMChain is deprecated.

LCEL Pattern chain = prompt | llm | output_parser

Use .invoke() for sync, .ainvoke() for async, .stream() for streaming

Use RunnableParallel for concurrent steps, RunnablePassthrough to pass input through

chain.with_retry(stop_after_attempt=3) for automatic retry on LLM API errors

2.3 Memory Strategies for Production

Memory Type	How It Works	Max Turns	Cost	Use Case	Avoid When
Buffer Memory	Stores all messages raw	~10-15 (context limit)	High token cost	Short demos only	Any production system
Window Memory	Last N messages only	Configurable	Medium	Simple chatbots	Need full history
Summary Memory	LLM summarizes old turns	Unlimited	LLM call per summary	Medium-length sessions	High-frequency chat (too many summaries)
Entity Memory	Tracks named entities	Unlimited	Medium	CRM bots, document agents	General Q&A
Vector Memory	Embeds + retrieves relevant history	Unlimited	Embedding cost	Long-running agents	Simple chatbots (overkill)
External DB (Redis/Postgres)	Store outside LLM context	Unlimited	Infra cost	Multi-user production	Single-user sessions

Production Memory Pattern Store conversation history in Redis or Postgres (keyed by session_id). Load only the last N turns + a running summary into the LLM context. Never store raw history in the LLM context for more than 5 turns.

2.4 Tool Calling — Deep Dive

How Tool Calling Works

11. You define tool schemas (name, description, parameters) alongside the LLM call
12. LLM decides IF and WHICH tool to call based on the query
13. LLM returns a structured tool_call object (not text)
14. Your code executes the actual tool
15. Tool result is appended to conversation and LLM generates final response

Critical Tool Design Rules

- Tool names and descriptions are PART OF THE PROMPT — write them like documentation
- Each tool should do ONE thing; avoid multi-purpose tools that confuse the model

- Always include: input validation, timeout, error message that helps the LLM recover
- Return structured data, not raw HTML — LLMs cannot parse raw web pages
- Log every tool call with inputs and outputs for debugging

ç Tool Calling Anti-Patterns No timeout on tool → hanging agents

Returning raw exceptions to LLM → model confusion/loops

Tool descriptions that overlap → model picks wrong tool

No retry on transient tool failures → flaky agents

Allowing tools to execute destructive actions without human-in-loop confirmation

2.5 LangChain vs Direct API Usage — Decision Guide

Scenario	Use LangChain	Use Direct API	Reason
Simple Q&A bot	No	Yes	LangChain overhead not worth it for single-call pipelines
RAG pipeline	Yes	Partial	LangChain retriever abstractions save significant work
Multi-step agent with tools	Yes	No	LCEL + LangGraph handle state + retry cleanly
High-throughput inference	No	Yes	LangChain adds latency overhead (50-200ms); direct SDK is faster
LLM switching (multi-model)	Yes	No	LangChain ChatModel abstraction makes model swaps trivial
Streaming responses	Both	Preferred	Direct SDK streaming is more predictable and lower latency
Custom tool integration	Yes	No	Tool schema + execution loop is painful to build from scratch
Fine-tuned model endpoint	No	Yes	LangChain integrations may lag behind custom endpoints

2.6 LangGraph — Stateful Graph-Based Agents

Core Concepts

- StateGraph: a directed graph where nodes are functions and edges are transitions
- State: a TypedDict that accumulates across all node executions
- Nodes: Python functions that read state, do work, return state updates
- Edges: deterministic (always) or conditional (based on state value)
- Checkpointing: save/restore graph state to Redis or Postgres for human-in-loop

When to Use LangGraph vs Plain LangChain

Pattern	LangGraph	Plain LCEL	Reason
Linear pipeline A→B→C	No	Yes	No branching = no need for graph overhead
Conditional routing (if/else)	Yes	Partial	LangGraph conditional_edges cleaner than nested runnables
Human-in-the-loop approval	Yes	No	LangGraph interrupt() + checkpointing built for this
Retry loops (agent re-tries)	Yes	No	Graph cycles enable natural retry without recursion bugs
Multi-agent orchestration	Yes	No	LangGraph handles agent handoff and message passing
Parallel branches	Yes	LCEL Parallel	Both work; LangGraph preferred for complex fan-out/fan-in

LangGraph Production Pattern Always use Checkpointer (AsyncRedisSaver or AsyncSqliteSaver) in production

Design state schema upfront — changing TypedDict schema breaks persisted checkpoints

Use interrupt_before=['human_review'] for any state that modifies external systems

Set recursion_limit on graph compilation to prevent infinite loops (default=25)

3. RAG Architecture — Standard to Agentic

3.1 RAG Pipeline — Complete Production Architecture

Naive RAG (Don't Use in Production)

Query → Embed → Vector Search → Top-K chunks → Stuff into LLM. Fails due to: poor retrieval quality, context stuffing, no grounding validation, no handling of out-of-scope queries.

Advanced RAG (Production Standard)

16. Query Analysis: classify query type, detect ambiguity, expand query
17. Query Transformation: HyDE (generate hypothetical answer to embed), multi-query, step-back prompting
18. Hybrid Retrieval: vector + BM25 + metadata filter
19. Re-ranking: Cohere/BGE cross-encoder on top-20
20. Context Compression: LLMLinguaCompressor to fit more into context window
21. Generation with citations: require LLM to cite chunk IDs
22. Grounding validation: verify answer is supported by retrieved context

3.2 CRAG — Corrective RAG

CRAG adds a retrieval evaluator that scores retrieved documents. If confidence is low, it triggers a web search correction step before generation.

- Evaluator: LLM-as-judge or fine-tuned classifier scores (relevant / partially relevant / irrelevant)
- Corrective step: if irrelevant → reformulate query + web search (Tavily, Serper, SerpAPI)
- Knowledge refinement: strip irrelevant parts of retrieved docs before passing to generator
- Best for: knowledge-intensive QA where retrieval can fail on obscure queries

RAG Variant	Retrieval	Validation	Self-Correction	Complexity	Best Use Case
Naive RAG	Single vector	None	None	Low	Prototyping
Advanced RAG	Hybrid + rerank	None	None	Medium	Most production
CRAG	Hybrid + rerank	LLM evaluator	Yes (web search)	High	High-stakes QA
Self-RAG	Adaptive retrieval	Critique tokens	Yes (regenerate)	Very High	Research quality
Agentic RAG	Multi-step planning	Agent decides	Yes (full loop)	Highest	Complex multi-source

3.3 Agentic RAG

Instead of fixed retrieval-then-generate, the agent plans which sources to retrieve from, retrieves iteratively, validates, and decides when it has enough context to answer.

- Planning: agent creates retrieval plan (which DBs, which queries, what order)
- Multi-source: retrieves from vector DB, SQL, APIs, web simultaneously
- Iterative: if first retrieval insufficient, agent refines and retrieves again
- Self-validation: agent checks if answer is grounded before responding

Agentic RAG Tradeoffs Latency: 3-10x slower than naive RAG due to multi-step reasoning

Cost: 5-20x more tokens due to planning + validation loops

Complexity: hard to debug, test, and monitor

Use only when: question complexity justifies it and users can tolerate 10-30s response times

3.4 Multi-Agent Architectures

Pattern	Description	LangGraph Support	Use Case
Supervisor	One orchestrator agent routes to specialist agents	Yes (conditional edges)	Complex multi-domain QA
Peer-to-peer	Agents communicate directly via message passing	Yes (graph nodes)	Collaborative research agents
Hierarchical	Multi-level supervisor trees	Yes (nested graphs)	Enterprise workflows
Swarm (handoff)	Agent hands off to next agent with context	Yes (interrupt + resume)	Customer service escalation
Map-Reduce	Parallel agents summarize, reducer synthesizes	Yes (parallel nodes)	Large document summarization

Agent Handoff — Implementation

- Use LangGraph's Command(goto='agent_name', update={'state': value}) for explicit handoff
- Pass full context in state — never assume receiving agent has memory
- Log handoff events: which agent, what triggered it, timestamp, session_id
- Implement handoff limits: max 5 handoffs per request to prevent infinite loops

3.5 OpenAI Agents SDK

- Agent class: wraps model + tools + instructions into a deployable agent
- Handoffs: native handoff mechanism between specialized agents
- Guardrails: input/output validation hooks built into the run loop
- Tracing: built-in OpenAI tracing dashboard (or export to LangSmith)
- Best for: teams standardized on OpenAI who want less boilerplate than LangGraph

4. Agent Protocols — A2A & MCP

4.1 Model Context Protocol (MCP)

MCP is an open protocol (Anthropic, 2024) that standardizes how LLM agents connect to external data sources and tools. Think of it as USB-C for LLM tool integrations.

Architecture

- MCP Host: the LLM application (Claude, your app)
- MCP Client: protocol client inside the host, manages connections
- MCP Server: exposes resources, tools, prompts over stdio or HTTP/SSE
- Resources: static or dynamic context (files, DB rows, API responses)
- Tools: callable functions with JSON schema definitions
- Prompts: reusable prompt templates with parameters

Comparison	MCP	Traditional Tool Calling	LangChain Tools
Standardization	Protocol (any LLM)	Model-specific	Framework-specific
Discovery	Dynamic (server advertises)	Static (hardcoded)	Static (registered)
Transport	stdio, HTTP/SSE	API calls	Python function calls
Cross-agent	Yes (shared servers)	No	No
Ecosystem	Growing rapidly	OpenAI-centric	LangChain-centric
Production maturity	Early (2024)	Mature	Mature

4.2 Agent-to-Agent (A2A) Communication

- Direct A2A: Agent A calls Agent B's API endpoint with context payload
- Pub/Sub A2A: Agents publish events to a message bus (Redis Streams, Kafka), others subscribe
- Orchestrated A2A: Supervisor agent dispatches tasks to worker agents and aggregates results
- Always include: correlation_id, originating_agent, task_id, timeout, max_retries in A2A calls
- Implement circuit breakers: if Agent B fails 3 times, return degraded response rather than loop

5. Securing LLMs & Guardrails

5.1 Threat Landscape

Attack	Description	Example	Mitigation
Prompt Injection	Malicious input overrides system prompt	User: 'Ignore previous instructions and...'	Input sanitization, strict system prompt, output validation
Indirect Injection	Malicious content in retrieved docs	Retrieved webpage contains injection	Sanitize retrieval context, separate user/system/retrieved namespaces
Jailbreak	Techniques to bypass safety alignment	DAN prompts, roleplay tricks, encoding	Input classifiers (Llama Guard), output filtering, refusal detection
Data Extraction	Tricking LLM to reveal training data or system prompt	'Repeat your system prompt'	Never put secrets in system prompt, response filtering
Model DoS	Sending complex prompts to maximize token usage	Deeply nested logic bombs	Token limits, rate limiting, cost alerts
Insecure Tool Execution	Agent executes harmful tool calls	Agent deletes files, makes purchases	Tool sandboxing, human-in-loop for destructive actions
Context Poisoning	Inserting false information into RAG context	Poisoned documents in vector DB	Document provenance validation, context filtering

5.2 Input Validation Pipeline

23. Length check: reject inputs exceeding max_input_tokens threshold
24. PII detection: scan for emails, phone numbers, SSNs before sending to LLM (use Presidio)
25. Injection pattern matching: regex for common injection prefixes
26. Content classification: run input through Llama Guard or similar safety classifier
27. Rate limiting: per-user, per-IP, per-session token budgets
28. Encoding attack detection: detect base64, hex-encoded, or unicode obfuscation

Microsoft Presidio Presidio is the go-to open-source library for PII detection and anonymization. Use analyzer.analyze() to detect, then anonymizer.anonymize() to mask before logging or storing conversations. Never log raw user messages to external services without PII scrubbing.

5.3 Output Filtering

- Toxicity scoring: Perspective API or Detoxify for harmful content detection
- PII leakage detection: scan LLM outputs for PII before returning to user
- Hallucination grounding check: verify claims are supported by retrieved context
- Refusal detection: detect if model refused and log for analysis
- Structured output validation: always use Pydantic to validate JSON outputs

5.4 Guardrail Design with NeMo Guardrails / Llama Guard

Tool	Type	What It Does	Latency Added	Best For
Llama Guard 3	Open-source classifier	Input + output safety classification	50-200ms	Multi-category safety
NeMo Guardrails	NVIDIA framework	Colang policy language for dialog rails	100-500ms	Enterprise policy compliance
OpenAI Moderation	API	Input content moderation	20-50ms	OpenAI-native apps
Lakera Guard	API	Prompt injection + jailbreak detection	50-150ms	Injection defense
Rebuff	OSS	Prompt injection heuristics + LLM detection	100-300ms	Self-hosted injection defense
Custom regex	Code	Pattern-based blocking	<1ms	Known patterns only

Guardrail Performance Warning Running all guardrails in series adds 500ms-1s+ to every request

Run input guardrails before LLM call, output guardrails after

For high-throughput: use async parallel guardrail evaluation

Cache guardrail results for identical inputs (short TTL)

5.5 Responsible AI — Human-in-the-Loop Controls

- High-stakes actions (delete, purchase, send email, API writes): ALWAYS require human approval
- Use LangGraph interrupt_before=['approval_node'] to pause and await human confirmation
- Implement confidence thresholds: if agent confidence <0.7, escalate to human
- Full audit trail: log every agent decision with rationale and input state
- Rollback capability: every agentic action should have a corresponding undo operation

6. GenAI System Evaluation & Quality Assurance

6.1 Evaluation Challenges

- LLMs produce variable outputs: same input, different outputs across runs
- Ground truth is expensive to create and goes stale as domain knowledge evolves
- Automatic metrics (BLEU, ROUGE) correlate poorly with human judgment for LLM outputs
- RAG evaluation requires separating retrieval quality from generation quality

6.2 Evaluation Framework Stack

Tool	Purpose	Metrics	Integration	Best For
DeepEval	RAG + LLM eval framework	Faithfulness, ContextRecall, RAGAS, G-Eval	LangChain, pytest	Production eval pipelines
RAGAS	RAG-specific evaluation	Context Precision/Recall, Answer Relevance, Faithfulness	Any RAG pipeline	RAG quality benchmarking
LangSmith	Tracing + eval	Custom LLM-as-judge, human annotation	Native LangChain	LangChain shops
Promptfoo	Prompt testing CLI	Custom metrics, safety, accuracy	Any LLM API	Prompt red-teaming
TruLens	Feedback functions	TruLens triad (groundedness, relevance, coherence)	Any framework	Research + production
Braintrust	Eval platform	LLM-as-judge, custom scorers	Any API	Team-based eval with UI
Arize Phoenix	Observability + eval	Embedding drift, retrieval quality	Many frameworks	Production monitoring + eval

6.3 RAG Evaluation — The Core Metrics

RAGAS Metrics (The Standard)

Metric	Measures	Formula Concept	Good Score	How to Improve
Faithfulness	Is answer grounded in context?	Verifiable claims / all claims in answer	>0.85	Better prompting to cite context; stronger re-ranking
Answer	Does answer	Cosine(generated_questions,	>0.80	Query

Metric	Measures	Formula Concept	Good Score	How to Improve
Relevance	address the question?	original_question)		understanding improvement; intent classification
Context Precision	Are retrieved chunks relevant?	Relevant chunks / all retrieved chunks	>0.75	Better chunking strategy; metadata filtering
Context Recall	Did we retrieve all needed info?	Needed info found / all needed info	>0.70	Expand k; hybrid search; query expansion
Answer Correctness	Is the factual answer right?	F1 between predicted and ground truth facts	>0.80	Better retrieval + stronger model
Context Entity Recall	Are key entities in context?	Retrieved entities / expected entities	>0.75	Improve chunking; add entity metadata

6.4 Golden Dataset — Building & Maintaining

- Minimum 100 QA pairs for each domain/topic area (200+ for reliable eval)
- Include: question, expected_answer, source_documents, expected_citations
- Diversity requirement: easy, medium, hard questions; single-hop and multi-hop reasoning
- Negative examples: questions the system should NOT answer (out-of-scope)
- Update cycle: add 20+ new examples per sprint as you discover failure modes
- Version control: store golden dataset in Git alongside prompt versions

Golden Dataset Creation Use LLM to generate initial QA pairs from your documents, then have domain experts validate

Include adversarial examples: ambiguous questions, questions with outdated info

Tag questions by difficulty and type — monitor metric breakdown by category

6.5 Hallucination Detection

Method	How	Speed	Accuracy	Cost
SelfCheckGPT	Generate N times; inconsistency = hallucination	Slow	Good	High (N x LLM calls)
NLI-based	Entailment check: claim supported by context?	Fast	Good	Low (local model)
LLM-as-Judge	Ask LLM to fact-check	Medium	Very Good	Medium

Method	How	Speed	Accuracy	Cost
	against context			
Semantic Similarity	Compare claim embedding to context embeddings	Very Fast	Moderate	Very Low
SAFE (Google)	Search-Augmented Factuality Evaluator	Slow	Excellent	High

6.6 LLM-as-Judge — Implementation

Use a capable LLM (GPT-4o, Claude Sonnet) to score outputs against criteria. More flexible than metrics, correlates well with human judgment when prompted correctly.

- Evaluation criteria must be explicit and specific — vague rubrics produce noisy scores
- Use 5-point Likert scale with anchored descriptions, not open-ended scores
- Include reasoning requirement: 'explain your score before giving it' (chain-of-thought improves accuracy)
- Calibrate against human labels: sample 5% of LLM-judge decisions and compare to human
- Avoid using the SAME model as both judge and generator (self-serving bias)

6.7 Regression Testing for Prompts

29. Every prompt change triggers full eval suite run
30. Compare new vs baseline on golden dataset: delta < -2% on any metric = reject
31. A/B test in shadow mode before full rollout
32. Track prompt versions with eval scores in LangSmith or Braintrust
33. Auto-block deployment if hallucination rate increases by >1%

7. LLMOps — Lifecycle, Deployment & Management

7.1 LLM Application Lifecycle

Phase	Activities	Key Tools	Gate Criteria
Development	Prompt engineering, RAG setup, agent design	LangChain, LangGraph, local Ollama	Eval score on golden set >threshold
Evaluation	Automated eval, red-teaming, safety checks	DeepEval, RAGAS, Promptfoo	All metric thresholds met; 0 critical safety failures
Staging	Integration testing, load testing, shadow mode	LangSmith, k6, staging vector DB	P95 latency <target; error rate <1%
Deployment	Canary → blue/green or rolling deploy	Docker, K8s, CI/CD pipeline	Canary metrics stable for 30min
Monitoring	Latency, cost, quality, drift monitoring	LangSmith, Arize, Prometheus	Automated alerts configured
Improvement	Root cause analysis, eval suite expansion, prompt iteration	LangSmith traces, DeepEval	Close feedback loop within SLA

7.2 MLOps vs LLMOps — Key Differences

Dimension	MLOps	LLMOps
Model versioning	Model weights + hyperparams	Model + prompt + retrieval index — all three must be versioned together
Evaluation	Accuracy on test set (deterministic)	LLM-as-judge + human eval (probabilistic)
Deployment unit	Model artifact	Model + prompt template + vector index + tool configs
Drift detection	Data distribution drift	Semantic drift in queries + quality drift in responses
Cost model	Training + inference compute	Tokens per request (input + output) + retrieval + guardrails
Rollback	Restore model checkpoint	Restore model version + prompt version + re-index vector DB
Testing	Unit + integration tests	Prompt unit tests + eval suite + adversarial red-team

7.3 Deployment Architectures

API-Based Deployment (Most Common)

- Stateless API service: each request carries full context; horizontally scalable
- Use FastAPI + LangServe for LangChain-native deployment with built-in /invoke, /stream, /batch
- Session state: Redis for conversation memory, Postgres for long-term persistence
- Vector DB: separate managed service (not co-located with inference service)

Hosted vs Self-Hosted Models

Option	Examples	Latency	Cost	Privacy	When to Choose
OpenAI API	GPT-4o, GPT-4o-mini	50-500ms	Pay-per-token	Data sent to OpenAI	Default for most; best quality
Anthropic API	Claude Sonnet/Opus/Haiku	50-500ms	Pay-per-token	Data sent to Anthropic	Long context, coding, safety-critical
Azure OpenAI	GPT-4o on Azure	50-500ms	Pay-per-token + infra	Data in your Azure tenant	Enterprise, data sovereignty
AWS Bedrock	Claude, Titan, Llama	50-500ms	Pay-per-token	Data in your AWS account	AWS shops, multi-model
Self-hosted (vLLM)	Llama 3, Mistral, Qwen	20-200ms	GPU infra cost	Full control	GDPR/HIPAA, cost at scale
Ollama (local)	Any GGUF model	100-2000ms	Free (local hardware)	Fully private	Dev, air-gapped environments

7.4 Fine-Tuning — When and How

- Fine-tune ONLY when: base model consistently fails on domain-specific format/style despite good prompting
- NOT for adding new knowledge — use RAG for knowledge. Fine-tune for behavior/format changes
- Data requirements: minimum 100 high-quality examples; 1000+ for meaningful improvement
- Always fine-tune smallest model that meets quality bar — not GPT-4 unless necessary
- Evaluate fine-tuned model against baseline on golden dataset before any deployment

Method	Parameter Efficiency	Quality	Use Case
Full Fine-tuning	Low (all weights)	Best	Small models (<7B), maximum customization
LoRA/QLoRA	High (adapters only)	Very Good	Large models, limited GPU memory
Prefix Tuning	High	Good	Specific prompt format adaption

Method	Parameter Efficiency	Quality	Use Case
RLHF	Low	Best for alignment	Safety, helpfulness alignment
OpenAI Fine-tuning API	N/A (managed)	Good	GPT-3.5/4o-mini quick fine-tuning

7.5 LiteLLM — Unified LLM Gateway

LiteLLM provides a unified OpenAI-compatible API over 100+ LLM providers. Use it as a gateway layer to get model routing, fallbacks, cost tracking, and load balancing.

- Fallback chain: [claude-3-5-sonnet, gpt-4o, llama-3-70b] — automatic failover
- Cost tracking: built-in per-request cost logging with budget limits
- Caching: semantic caching with Redis reduces duplicate LLM calls
- Load balancing: round-robin or least-latency across multiple API keys
- Proxy mode: deploy as a centralized gateway — teams use one endpoint, you control routing

LiteLLM Production Setup Deploy LiteLLM proxy behind your API gateway

Set budget_limit per team/project in config.yaml

Enable success_callback=['langsmith'] for automatic tracing

Use router_strategy='latency-based-routing' for optimal performance

7.6 Versioning Strategy

Component	What to Version	Where to Store	Rollback Strategy
Prompt Templates	Every change with eval scores	Git (prompts/ directory) + LangSmith	Revert Git commit + redeploy
LLM Model	Model name + version + temperature	Config file in Git	Change config + redeploy (stateless)
Embedding Model	Model name + dimension	Pinned in requirements.txt + DB metadata	Re-index all documents (expensive — avoid)
Vector Index	Snapshot of indexed data	S3 snapshot or Pinecone namespacing	Restore snapshot or switch namespace
RAG Pipeline	Full pipeline config (chunk size, k, reranker)	Git + semantic versioning tags	Deploy previous tagged release

Component	What to Version	Where to Store	Rollback Strategy
Agent Graph	LangGraph compiled graph definition	Git	Revert graph definition + redeploy

7.7 Cost Optimization

- Prompt caching: Anthropic and OpenAI support prompt caching for repeated system prompts (up to 90% savings on cached tokens)
- Semantic caching: LiteLLM + Redis caches semantically similar queries (configure similarity threshold ~0.95)
- Model routing: use cheap models (GPT-4o-mini, Haiku) for simple tasks; expensive models for complex reasoning
- Context management: trim conversation history aggressively; don't pass irrelevant chunks
- Batch API: for non-real-time tasks use OpenAI Batch API (50% cost reduction with 24h latency)
- Output length control: set max_tokens tightly — verbose outputs cost money

Optimization	Savings	Latency Impact	Complexity
Prompt caching	50-90% on repeated prompts	Neutral to positive	Low — just structure prompts correctly
Semantic caching	20-60% depending on query repetition	Reduction (cache hits)	Medium — need Redis + embedding similarity
Model downgrade	50-80% per token	May increase (worse models retry more)	Low — easy config change, test quality first
Context compression	30-60% token reduction	Slight increase (compression step)	Medium — LLMLingua or custom summarizer
Batch API	50% off	High (24h delay)	Low — only for offline workloads

8. Observability & Monitoring

8.1 LLM Observability Stack

Tool	Primary Use	What It Captures	Self-hosted	Best For
LangSmith	Tracing + debugging	Full LangChain run trees, prompts, outputs, latency	No (Anthropic-owned)	LangChain-native teams
Arize Phoenix	Production monitoring + eval	Embedding drift, retrieval quality, hallucination	Yes (OSS)	Multi-framework production
Helicone	LLM proxy observability	Cost, latency, user analytics, caching	Yes/Cloud	Cost monitoring, rate limiting
Langfuse	Tracing + eval platform	Traces, scores, dataset management	Yes (OSS)	Self-hosted alternative to LangSmith
Weights & Biases (W&B)	Experiment tracking	Eval runs, prompt experiments, model metrics	No	Research + eval tracking
Prometheus + Grafana	Infrastructure metrics	Latency histograms, error rates, throughput	Yes	Custom infra dashboards
OpenTelemetry	Distributed tracing	Cross-service request tracing	Yes	Multi-service architectures

8.2 What to Log — The Minimum Viable Observability

Every LLM Request Must Log

- request_id, session_id, user_id (anonymized), timestamp
- model_name, model_version, prompt_version, pipeline_version
- input_tokens, output_tokens, total_cost (calculated)
- latency_ms (time to first token + total), retry_count
- retrieved_chunk_ids and similarity scores (for RAG)
- tool_calls made (name, inputs, outputs, duration)
- output_text (with PII redacted), finish_reason, safety_flags
- user_feedback (thumbs up/down, corrections) — async, linked by request_id

ç Never Log in Production Raw user messages without PII scrubbing

Full system prompts (security risk — reveals business logic)

API keys or credentials (even in encrypted logs)

User-identifying information without explicit consent and legal basis

8.3 Key Metrics & Alerting Thresholds

Metric	Target	Warning	Critical	Alert Action
P50 Latency	<1s	1-2s	>3s	Page on-call
P95 Latency	<3s	3-5s	>8s	Page on-call + auto-scale
P99 Latency	<8s	8-15s	>20s	Circuit break + escalate
Error Rate	<1%	1-3%	>5%	Page on-call + rollback ready
Token Cost / Day	<budget	80% of budget	100% budget	Hard stop + notify finance
Faithfulness Score	>0.85	0.75-0.85	<0.75	Alert team + review pipeline
Cache Hit Rate	>30%	15-30%	<15%	Review semantic cache config
Hallucination Rate	<5%	5-10%	>10%	Halt new deployments + investigate

8.4 Detecting Drift in Production

Query Distribution Drift

- Embed incoming queries weekly and compare to baseline embedding distribution
- Use Maximum Mean Discrepancy (MMD) or Jensen-Shannon divergence as drift metrics
- Trigger re-evaluation of golden dataset when drift detected

Response Quality Drift

- Run LLM-as-judge on 5% random sample of production responses daily
- Track rolling 7-day average of quality scores
- Alert when 7-day average drops >5% relative to baseline

Retrieval Drift

- Monitor average cosine similarity of top-retrieved chunks to query
- Sudden drop in average similarity = documents changed or embedding model issue
- Track context precision on evaluation queries weekly

8.5 Error Analysis & Debugging Workflows

Error Pattern	Likely Root Cause	Debug Steps	Fix
Empty or nonsensical answers	Retrieval failure — no relevant chunks found	Check top-k similarity scores in LangSmith trace; look at retrieved content	Tune similarity threshold; improve chunking; add query expansion
Hallucinated facts not in context	Generation ignoring context	Check if context was passed correctly; review prompt structure	Strengthen grounding instruction in prompt; add NLI validator
Correct info but wrong format	Output parser failure	Check raw LLM output before parsing in trace	Add format examples to prompt; use stricter output parser
Agent stuck in loop	Max iterations hit or circular tool calls	Inspect agent trace step-by-step in LangSmith	Add loop detection; increase recursion limit; improve tool descriptions
High latency on simple queries	Re-ranker or guardrail bottleneck	Profile each step's latency in trace	Async guardrails; cache frequent patterns; remove unnecessary steps
Cost spike	Prompt growing unbounded	Check input_tokens trend; look for memory leak	Implement conversation summarization; trim context aggressively

9. Deployment, Versioning & Scalability

9.1 Deployment Pipeline — End-to-End

34. Dev: build + prompt engineer + local eval (DeepEval on golden set)
35. PR opened: automated eval suite runs (CI GitHub Actions + DeepEval)
36. Code review: peer review of prompt changes, agent logic, tool additions
37. Staging: deploy to staging, run full integration tests + load test (k6)
38. Canary: deploy to 5% of traffic, monitor quality + latency + errors for 30min
39. Rollout: gradual increase to 25% → 50% → 100% over 2-4 hours
40. Monitor: watch dashboards actively for 1 hour post full rollout

9.2 Scalability Patterns

Challenge	Solution	Tradeoff	Implementation
High throughput requests	Horizontal scaling behind load balancer	Cost increases linearly with replicas	Kubernetes HPA on CPU/latency metrics
Slow LLM responses	Streaming + async processing	UX shows partial results; complex client code	FastAPI StreamingResponse + SSE
Rate limits on LLM API	Multiple API keys + load balancing	Key management complexity	LiteLLM router with multiple keys
Repeated expensive queries	Semantic + exact caching	Stale cache risk; cache invalidation complexity	Redis with TTL + LiteLLM cache
Vector DB overload	Read replicas + connection pooling	Write propagation delay to replicas	Qdrant/Pinecone replica config
Agent timeouts	Async execution + webhook callback	Complex client handling	FastAPI background tasks + status polling

9.3 Environment Management

- dev: local or shared dev cluster; permissive limits; debug logging enabled
- staging: production-mirror with synthetic data; full guardrails; eval suite runs
- production: locked configs; minimal logging verbosity; cost controls enforced
- Never share API keys across environments — each environment has its own key set
- Use environment variables for ALL credentials — never hardcode (12-factor app)
- Infrastructure as Code: all K8s configs, env configs in Git (GitOps pattern)

9.4 Inference Pipeline Management

Request Preprocessing

- Sanitize and validate input format and length
- Inject user context (permissions, preferences, history summary)
- Route to appropriate pipeline based on query classification
- Apply rate limiting and quota checks

Context Assembly

- Fetch conversation history from Redis (last N turns)
- Run retrieval pipeline → top-k chunks with re-ranking
- Assemble context window: system prompt + history + retrieved context + user query
- Measure assembled token count — truncate if approaching model limit

Post-processing

- Validate output format (Pydantic parsing)
- Run output safety filters
- Strip internal chain-of-thought if not meant for user
- Log full request/response pair with metadata
- Return streaming or complete response

9.5 Rollback Strategies

What Changed	Rollback Method	Time to Rollback	Risk
Prompt only	Revert Git commit + redeploy container	5-15 min	Low — stateless
LLM model version	Change model config + redeploy	5-15 min	Low — if API-hosted
Agent graph/logic	Revert code + redeploy	10-30 min	Medium — test in staging first
Vector index (re-indexed)	Restore S3 snapshot or switch Pinecone namespace	30-120 min	High — snapshot freshness
Embedding model change	Re-index all documents with old model	Hours to days	Very High — always plan this carefully

10. Good Practices Before Pushing to GitHub

10.1 Pre-Push Checklist

Security — Never Push These

ç Absolute Rules Before Every Git Push Run: git diff --staged | grep -i 'api_key\|secret\|password\|token' to catch credential leaks

Never commit .env files — only .env.example with placeholder values

Check for hardcoded model names with version numbers that should be in config

Scan with gitleaks or trufflehog: gitleaks detect --source . --verbose

Ensure .gitignore includes: .env, *.env, __pycache__, .chroma/, *.faiss, vector_store/

Code Quality Checks

41. Run pre-commit hooks: pre-commit run --all-files
42. Black/ruff for Python formatting and linting
43. mypy for type checking on core modules
44. Unit tests must pass: pytest tests/ -v --tb=short
45. Eval suite on golden set: deepen ci run or similar eval CI command
46. Check token budget changes: compare prompt lengths before/after your change

10.2 Prompt Change Protocol

Action	Required Before Push	Reviewer Needed	Documentation
Minor wording fix	Run golden set eval — delta <1%	No	Update CHANGELOG.md
System prompt change	Full eval suite + safety check	Yes (1 reviewer)	Prompt version bump + eval scores in PR
New tool added	Tool unit test + integration test + eval	Yes (1 reviewer)	Tool documentation in tools/README.md
Retrieval config change	RAG eval on golden set + A/B in staging	Yes (tech lead)	Config version bump + benchmark comparison
Embedding model change	Full re-index plan + rollback plan documented	Yes (2 reviewers)	Migration runbook required
Agent graph restructure	Full integration test + failure mode analysis	Yes (2 reviewers)	Architecture diagram update required

10.3 Repository Structure (Recommended)

ō Recommended Project Structure prompts/ — versioned prompt templates (v1.0.0.yaml, v1.1.0.yaml)

evals/ — golden datasets, eval configs, eval runner scripts

agents/ — LangGraph graph definitions, agent configs

tools/ — tool implementations + unit tests + README

retrieval/ — chunking, embedding, retrieval pipeline code

guardrails/ — input/output safety filters

monitoring/ — dashboards, alert configs, runbooks

tests/ — unit, integration, smoke tests

scripts/ — deployment scripts, index management

docs/ — architecture diagrams, runbooks, ADRs

.github/workflows/ — CI pipelines: eval, test, lint, deploy

10.4 CI/CD Pipeline Configuration

- On every PR: lint + type check + unit tests + prompt eval on golden set
- On merge to main: full eval suite + staging deploy + integration tests
- On release tag: canary deploy + monitoring alerts + rollback script ready
- Quality gates: PR blocked if any metric regresses by >2% on eval suite
- Cost gate: PR blocked if estimated prompt cost increase >20% with no approval

10.5 ADR — Architecture Decision Records

Every significant technical decision (embedding model choice, vector DB selection, agent framework, guardrail tool) should have an ADR in docs/adr/. Include: Context, Decision, Alternatives Considered, Consequences, Date, Author.

11. Quick Decision Guide — What to Choose When

11.1 Choosing Your Vector Database

Your Situation	Choose	Reason
Production, team, managed, no infra ops	Pinecone	Zero-ops, reliable, scales to billions
Production, need rich filtering, metadata queries	Qdrant	Best payload filtering, ANN + filter combo
Production, multi-modal (images + text), GraphQL queries	Weaviate	Built for multi-modal and graph-like queries
Already on Postgres, small-medium scale (<5M vectors)	pgvector	No new infra; SQL ecosystem
Development/prototype only	ChromaDB	Easiest local setup; not for production
Research, custom pipeline, maximum control	FAISS	No vendor, maximum flexibility, no persistence built-in

11.2 Choosing Your LLM

Task	Best Model	Alternative	Avoid
Complex reasoning, coding, long docs	Claude Sonnet 3.7 / GPT-4o	Gemini 1.5 Pro	GPT-4o-mini for complex tasks
High-throughput, cost-sensitive, simple QA	GPT-4o-mini / Claude Haiku	Llama 3 8B (self-hosted)	GPT-4o (overkill)
Structured JSON output	GPT-4o with JSON mode	Claude with tool use	Smaller models (unreliable JSON)
Long context (100K+ tokens)	Claude Sonnet (200K)	Gemini 1.5 Pro (1M)	GPT-4o (limited context)
Code generation	Claude Sonnet 3.7 / GPT-4o	Codestral, DeepSeek Coder	Generic small models
Data privacy / air-gapped	Llama 3 70B (self-hosted)	Mistral Large (Azure)	Any external API

11.3 Choosing Your Embedding Model

Scenario	Model	Dimensions	Notes
Default production (quality)	text-embedding-3-large (OpenAI)	3072 (or reduce to 1536)	Best English quality; costly at scale
Default production (balanced)	text-embedding-3-small (OpenAI)	1536	Good quality, 5x cheaper than large
Self-hosted, free, high quality	BGE-M3 (BAAI)	1024	Multilingual, top-tier open model
Multilingual production	multilingual-e5-large	1024	Strong multilingual, free
Domain-specific (legal/medical)	Fine-tuned BGE or E5	1024	Requires fine-tuning data
Ultra-fast, RAM-limited	all-MiniLM-L6-v2	384	Fast but lower quality

11.4 Choosing Your Evaluation Tool

Need	Tool	Why
RAG quality metrics (standard)	RAGAS or DeepEval	Industry standard RAG metrics; easy integration
LangChain pipeline tracing + eval	LangSmith	Native integration; best debugging experience
Self-hosted eval platform	Langfuse	OSS; full control; good UI
Prompt red-teaming and safety	Promptfoo	CLI-based; great for adversarial testing
Production monitoring + drift	Arize Phoenix	Embedding drift detection; production monitoring
Human annotation + team eval	Braintrust	Great UI for human-in-the-loop scoring

12. What to Avoid — Anti-Patterns Reference

12.1 RAG Anti-Patterns

ç Top RAG Mistakes in Production Naive chunking (fixed 512 chars) — destroys semantic coherence across sentence boundaries

Not storing chunk metadata — you cannot filter, cite, or debug without it

Using cosine similarity alone — always hybrid with BM25 for exact-match resilience

Ignoring chunk overlap — set 10-15% overlap to prevent context loss at boundaries

Retrieving top-3 chunks when you need top-20 for re-ranking — starves the re-ranker

Not evaluating retrieval separately from generation — you cannot fix what you cannot measure

Using the same vector DB for dev and prod — dev data pollutes prod retrieval

12.2 Agent Anti-Patterns

ç Top Agent Mistakes No max_iterations limit — agents loop forever and burn tokens + money

Tools with side effects (write, delete, send) without human-in-loop confirmation

Giving agents too many tools (>10) — model gets confused; precision drops sharply

No timeout on tool calls — one hanging tool blocks the entire agent

Passing full conversation history to every agent call — context window bloat

No fallback when all tools fail — agent should degrade gracefully, not crash

Testing agents only with happy-path inputs — test failure modes explicitly

12.3 Security Anti-Patterns

ç Security Mistakes That Get Teams Fired Putting secrets or PII in system prompts — trivially extractable by adversarial users

Trusting LLM output for security decisions — never use LLM output to authorize actions

No rate limiting — open to prompt flooding / cost-DoS attacks

Logging raw user messages to third-party services without PII scrubbing

Using eval() or exec() on LLM-generated code without sandboxing

No input length limit — long inputs can exploit context confusion

Allowing agent to make external API calls without allowlist

12.4 LLMOps Anti-Patterns

ç LLMOps Mistakes That Break Production Changing embedding model without re-indexing all documents (silent retrieval failure)

No prompt versioning — cannot rollback after regression

Deploying on Friday afternoon (obvious but still happens)

No cost alerts — a bug can cost thousands in tokens overnight

No golden dataset — you're flying blind on quality

Only testing happy path — production is adversarial

Manual deployments — no audit trail, no rollback, no consistency

13. Tradeoff Reference Guide

13.1 The Core Tradeoffs Every Engineer Faces

Decision	Option A	Option B	Choose A When	Choose B When
Retrieval k size	Small k (top-3-5)	Large k (top-15-20)	Low latency critical; context window tight	Re-ranker available; diverse answers needed
Chunking size	Small chunks (128-256 tokens)	Large chunks (512-1024 tokens)	High precision retrieval needed	Preserving document context important
Model size	Small/cheap model	Large/powerful model	High throughput; cost-sensitive; simple tasks	Complex reasoning; quality-critical; low volume
Exact vs semantic cache	Exact match cache	Semantic similarity cache	Queries very repetitive verbatim	Queries vary in phrasing but same intent
Sync vs async agent	Synchronous (wait for result)	Async (poll/webhook)	Simple UI; latency <5s acceptable	Long-running tasks; complex agent workflows
Self-hosted vs managed LLM	Self-hosted (vLLM, Ollama)	Managed API	Data sovereignty; very high volume; privacy	Default; quality priority; small team
Human-in-loop	Require approval	Fully automated	Destructive/irreversible actions; high stakes	Read-only, low-stakes, reversible actions

14. Complete End-to-End Production Pipeline

14.1 Full System Architecture Summary

This section describes the complete pipeline from user query to response in a production-grade RAG + Agent system.

Layer	Component	Technology Choices	Monitoring
Ingress	API Gateway + Auth	Kong, AWS API GW, nginx	Request rate, auth failures
Rate Limiting	Per-user token budget	Redis + custom middleware	Budget consumption rate
Input Safety	PII detection + injection check	Presidio + Lakera Guard	Blocked request rate
Session Management	Conversation history	Redis (TTL=24h) + Postgres (long-term)	Session size, memory usage
Query Analysis	Intent classification + routing	Fine-tuned classifier or LLM router	Classification accuracy
Retrieval	Hybrid search + re-ranking	Qdrant/Pinecone + Cohere Rerank	Context precision, latency
Orchestration	Chain/agent execution	LangGraph + LCEL	Step latency, tool call rate
LLM Inference	Model API calls	LiteLLM proxy → OpenAI/Anthropic	Token cost, latency, errors
Output Safety	Content filter + PII scan	Llama Guard + Presidio	Flagged output rate
Response Delivery	Streaming or batch response	FastAPI SSE or JSON	P95 latency, error rate
Tracing	Full request trace	LangSmith / Langfuse	Trace completeness
Evaluation	Async quality scoring	DeepEval scheduled jobs	Faithfulness, relevance trends
Alerting	Threshold-based + anomaly	Prometheus + PagerDuty	Alert fatigue rate, MTTD

14.2 The Engineer's Daily Reference Card

Daily Production Checks 1. Check cost dashboard — any unexpected spikes?

2. Review P95 latency trend — increasing means retrieval or model degradation
3. Check error rate dashboard — any new error patterns?
4. Review safety flag rate — uptick may indicate new attack pattern
5. Check eval score rolling average — trending down = investigate retrieval or model drift

Weekly Tasks 1. Run full golden dataset eval — compare to previous week baseline

2. Review top 20 failed/low-scored queries — identify common failure modes
3. Check vector index health — any documents not indexed? Stale documents?
4. Review token cost breakdown — optimize any wasteful pipeline steps
5. Check LangSmith/Langfuse traces for slow steps — profile and optimize

End of LLMOps Field Reference Guide